Case study by Chandana Vishnu Jayanth · vishnuatgit.github.io · jayanthvishnu9@gmail.com
Live demo: archlens-xhxr.onrender.com · Code: github.com/vishnuatgit/ArchLens

ArchLens

Scores any public GitHub repository from 0 to 100 across seven engineering dimensions and checks its dependencies for known vulnerabilities, as a web app, a REST API, a CLI and a GitHub Action.

YEAR

2026

BUILT WITH

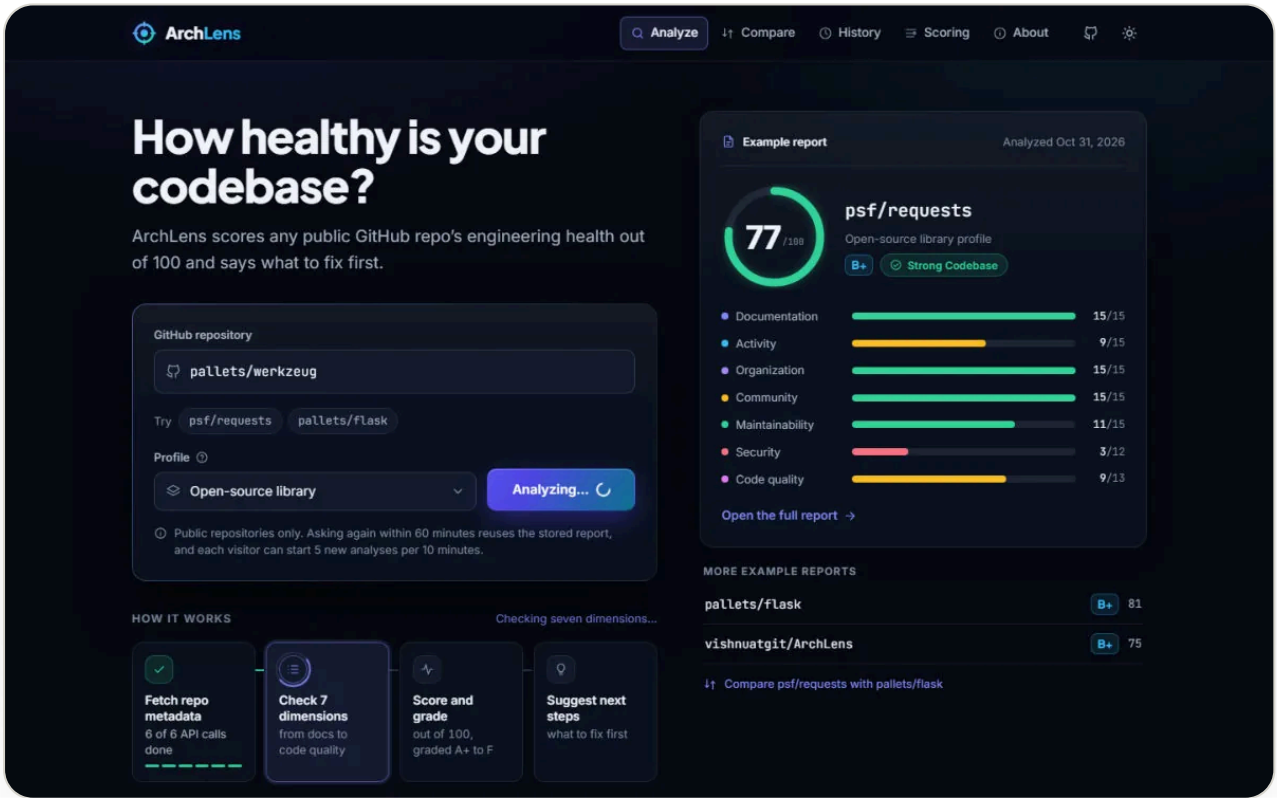
FastAPI · SQLAlchemy · Docker

PROOF

- 328 tests · CI passing

SOURCE

github.com/vishnuatgit/ArchLens



(01) The problem

WHY IT EXISTS

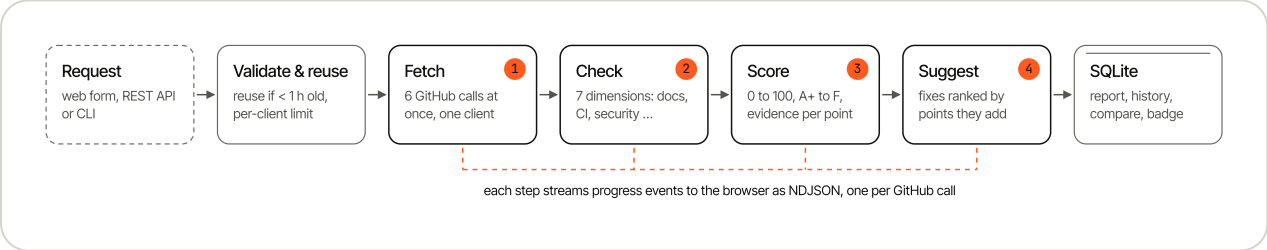
ArchLens checks a repository's *engineering health* in one request.

By hand, you would open the README, look for a license, tests and CI, and scan recent commits and issues. ArchLens does those checks at once, shows what it found for each, and lists what is missing as fixes, most valuable first.

It reads repository metadata through the GitHub REST API and never clones the code, so an analysis takes six read-only API calls. Every new analysis is stored in SQLite, so reports can be revisited, compared and shared.

(02) How it works

FROM THE README



1 **Fetch repo metadata.** Repository stats, the root file listing, languages, contributors, commits from the last 30 days and the workflow folder, read with one shared `httpx.AsyncClient` and `asyncio.gather`. If one call fails, the others are cancelled.

2 **Check 7 dimensions.** Documentation, activity, organization, community, maintainability, security and code quality: a README, license, test folder, CI workflows, lockfile, `SECURITY.md`, linter and type-checker config, recent commits and open issues.

3 **Score and grade.** Points add up to 100 and map to a grade from A+ (95 and above) to F (below 35) and a health tier, with the evidence behind every point.

4 **Scan dependencies.** Root manifests (`requirements*.txt`, `pyproject.toml`, `package.json` with its lockfile, `go.mod`) are read through the contents API, and pinned versions are checked against OSV.dev in one batch request: severity, advisory and the version to upgrade to. If OSV is down, the report says it couldn't check, and the score formula is unchanged.

5 **Suggest next steps.** Each failed check becomes a fix with the points it would add, ranked, and exportable as a Markdown checklist.

6 **Gate pull requests.** A composite GitHub Action runs the CLI in CI, writes the report to the job summary, keeps one updated PR comment and fails the job below a chosen score.

(03) Key decisions

6 CHOICES

01 One shared async client, metadata only

The six GitHub calls run concurrently through one `httpx.AsyncClient` shared for the app's lifetime. If one fails (a 404 on the metadata call, say), the calls still running are cancelled instead of spending more GitHub quota.

02 Upstream errors become clear responses

One error shape across the web app, the API and the CLI: a missing repository is a `404 REPO_NOT_FOUND`, a used-up GitHub quota a `503 RATE_LIMIT_EXCEEDED`, an unreachable GitHub a `502 GITHUB_UNAVAILABLE`, and too many analyses a `429` with `Retry-After`.

03 Cache and rate-limit for a free host

A report of the same repository and profile from the last hour is reused, and each client can start 5 new analyses per 10 minutes. An analysis that fails because GitHub is down or out of quota is given back.

04 Progress from real events

`POST /api/v1/analyze/stream` streams the pipeline's progress as newline-delimited JSON, one event per GitHub call, so the steps on the home page light up from what the server is actually doing, and a failed step says why.

05 Scoring you can check

The `/scoring` page is generated from the scorer's own constants, and tests run the real scorer against it, so the explanation cannot drift from the code.

06 Measured under load, then fixed

A Locust test against seeded reports found 30 s stalls at 50 users: async handlers made blocking database calls, so requests waiting for a pooled connection blocked the event loop. Read-only handlers became plain `def` and SQLite moved to `NullPool`. At 50 users p95 is now 510 ms with 0 failures (one laptop run; the README has the method).

(04) Proof

TESTS · CI · STACK

328

PYTEST TESTS, GITHUB MOCKED, NO NETWORK

Passing

CI ON GITHUB ACTIONS: PYTEST ON SQLITE AND POSTGRESQL, RUFF, BLACK

6

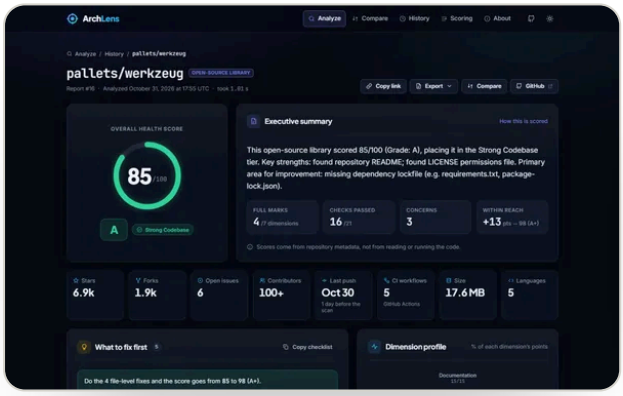
GITHUB API CALLS PER ANALYSIS, RUN CONCURRENTLY

Unit tests cover the scorer, the scoring page, the evidence (the real scorer over about 900 generated repositories), the fix plan, the progress events, the GitHub client and the rate limiter. Integration tests drive every page and API endpoint, including the progress stream, through FastAPI's `TestClient`. Dependency scanning is tested end to end with fake GitHub and OSV, and CI also runs the suite on PostgreSQL 16. CI runs ↗

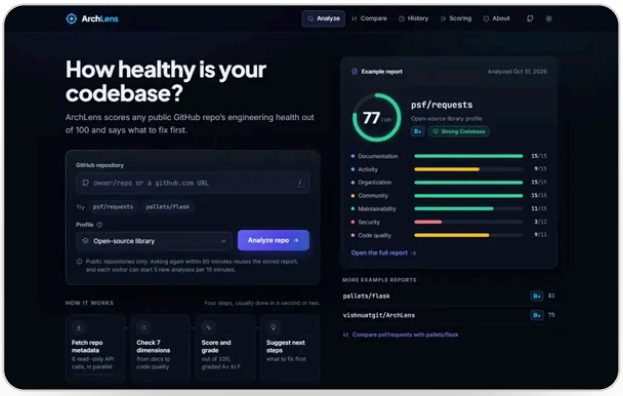
`Python 3.11+` `FastAPI` `httpx` `SQLAlchemy 2` `Alembic` `SQLite` `PostgreSQL` `OSV.dev API` `Jinja2` `Typer + Rich` `pytest` `Locust` `Ruff` `Black` `Docker` `GitHub Actions`

(05) Screenshots

FROM THE REPOSITORY



A report: the score and grade, the summary, and the facts behind them.



The home page: the steps light up as the server streams its progress.

(06) Limitations

HONESTLY

— **Metadata only.** It reads repository statistics, the root file listing and root dependency manifests. It does not read source files, run tests or measure coverage, and it checks direct, pinned dependencies only.

— **Simple rules.** A file earns points just by existing, whatever its quality, and stars say little about code health. Treat a low score as a list of things to check.

— **Capped counts.** Contributors and recent commits come from one page of GitHub results, so they stop at 100.

— **Single instance.** The rate limiter lives in memory and resets on restart.

— **Ephemeral demo storage.** On the free host, reports other than the bundled examples disappear when the service restarts.

NEXT CASE STUDY

DeepBell ↗

A mini SRE lab in FastAPI: it detects anomalies in its own CPU and memory, writes incident reports and restores from snapshots.